UNIVERSIDAD INTERNACIONAL DEL ECUADOR

Ingeniería en Sistemas - Tercer semestre

PROYECTO INTEGRADOR

IMAGEFLOW

Sistema cliente-servidor para la gestión de imágenes

SEGUNDO ENTREGABLE - SEMANA 10

Asignatura	Programación Orientada a Objetos
Estudiante	Sebastián Alexis Cumbal Molineros
Modalidad	Desarrollo individual
Docente	Por completar
Fecha	9 de noviembre de 2026

Documento técnico del prototipo funcional desarrollado en Go

1. Introducción y contexto

ImageFlow es un prototipo de aplicación empresarial orientado a gestiónar imágenes desde un cliente de consola hacia un servidor central. La aplicación separa las responsabilidades de presentacion, comunicación HTTP, procesamiento de solicitudes, reglas de negocio, modelo de datos y almacenamiento.

El segundo entregable corresponde a la semana 10 de Programación Orientada a Objetos. La guía solicita una primera versión ejecutable en Go, organizada modularmente y demostrable mediante procesos independientes de cliente y servidor.

2. Objetivo del prototipo

Desarrollar una aplicación cliente-servidor en Go que permita cargar, registrar, consultar, buscar, descargar y eliminar imágenes, aplicando estructuras, constructores, métodos, encapsulamiento, composición, modularización, validación de datos y manejo explícito de errores.

3. Alcance y funcionalidades implementadas

Operación	Descripción
Carga	El cliente envía una imagen al servidor mediante HTTP y multipart/form-data.
Metadatos	Se registran ID, nombre, tipo MIME, tamaño, categoría y fecha de creación.
Consulta	El cliente solicita la lista de imágenes registradas.
Búsqueda	Se filtra por coincidencia parcial en nombre y/o categoría.
Descarga	El cliente recupera el archivo original por medio de su identificador.
Eliminación	Se elimina el archivo y su registro de metadatos, con confirmación en el cliente.
Validación	Se aceptan JPEG, PNG, GIF y WEBP, con límite de 10 MiB por imagen.

4. Decisiones técnicas

- Lenguaje de programación del cliente y del servidor: Go 1.22 o superior.
- Dependencias externas: ninguna; se utiliza la biblioteca estándar de Go.
- Comunicación: HTTP, con respuestas JSON y carga de archivos multipart/form-data.
- Persistencia local: archivos binarios en data/images/ y metadatos en data/metadata.json.
- Identificadores: valores hexadecimales aleatorios generados en el servidor.

4.1 Posición técnica del autor

La decisión de diseño fue priorizar un prototipo pequeño, modular y fácil de demostrar, en lugar de introducir componentes que no son necesarios para comprobar los objetivos de la semana 10. Por eso se eligió un cliente de consola y una API HTTP: permiten observar claramente que el cliente y el servidor son procesos independientes y que las operaciones viajan por la red.

- El uso de archivos y JSON simplifica la persistencia y evita dependencias externas; como contrapartida, no ofrece las consultas, transacciones y administración de una base de datos, por lo que es una solución apropiada para el prototipo y no para una instalación empresarial de producción.
- El servidor detecta el tipo MIME a partir del contenido y genera nombres internos aleatorios. Esto evita confiar solamente en la extensión o en la ruta indicada por el cliente; no sustituye un análisis antivirus ni una validación profunda de todos los formatos.
- La autenticación no se duplica dentro del programa Go en esta fase. Se mantiene pendiente para la integración prevista por la guía académica, lo que evita desarrollar dos mecanismos de identidad diferentes.

5. Diagrama de componentes y arquitectura

La aplicación se organiza en dos procesos ejecutables: el cliente Go y el servidor Go. El cliente no modifica directamente el almacenamiento del servidor; envia solicitudes HTTP y recibe respuestas. El servidor coordina los paquetes internos para procesar cada operación.

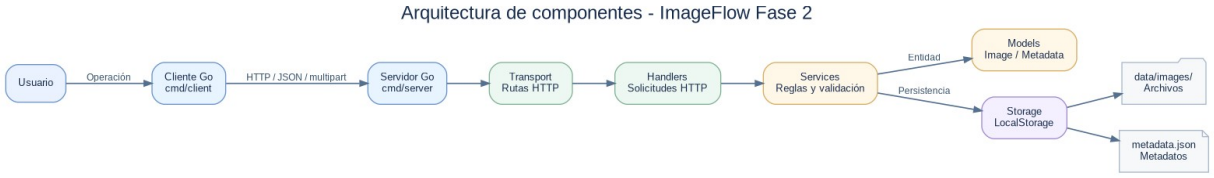


Figura 1. Arquitectura modular del prototipo ImageFlow.

6. Comunicación cliente-servidor

El servidor escucha por defecto en 0.0.0.0:8080. El cliente se conecta por defecto a http://localhost:8080. Ambos valores se pueden cambiar mediante variables de entorno, por lo que los componentes pueden ejecutarse en terminales distintas e incluso en equipos separados dentro de una red de laboratorio.

Metodo	Ruta	Función
GET	/health	Comprueba que el servidor está disponible.
POST	/api/v1/images	Recibe una imagen y su categoría mediante multipart/form-data.
GET	/api/v1/images	Devuelve metadatos; admite filtros name y category.
GET	/api/v1/images/{id}/download	Entrega el archivo original identificado por ID.
DELETE	/api/v1/images/{id}	Elimina el archivo y sus metadatos.

Flujo de carga

1. El usuario elige un archivo en el menú del cliente.
2. APIClient crea la solicitud HTTP POST con el archivo y la categoría.
3. ImageHandler recibe el multipart, limita el tamaño y delega al servicio.
4. ImageService valida nombre, tamaño y tipo de contenido; luego construye el modelo Image.
5. LocalStorage escribe el archivo y actualiza metadata.json.
6. El servidor responde con los metadatos y el cliente confirma la operación.

7. Estructuras, constructores y métodos

Go no utiliza clases e herencia como otros lenguajes orientados a objetos. En este prototipo se aplican los fundamentos de POO mediante structs, métodos, encapsulamiento por campos privados, constructores, composición y paquetes.

Componente	Responsabilidad y elementos de POO
models.Image	Entidad principal. Sus campos son privados. NewImage valida datos antes de crear el objeto. Los métodos ID(), Name(), Size(), Category() y Metadata() exponen la información necesaria.
models.Metadata	Estructura pública para intercambiar los datos visibles mediante JSON.
models.Record	Compone Metadata e incorpora StoredName para la persistencia interna.
storage.LocalStorage	Mantiene un mapa de registros y un sync.RWMutex. NewLocalStorage prepara y carga el almacenamiento; Save, List, Get, ReadFile y Delete gestiónan los datos.
services.ImageService	Contiene la logica de negocio. Upload, List, Download y Delete validan y coordinan operaciónes.
handlers.ImageHandler	Recibe solicitudes HTTP y traduce errores de negocio a códigos HTTP y mensajes JSON.
client.APIClient	Agrupa las operaciónes HTTP que utiliza el menu del cliente.

Constructores y composición

- `NewImage(...)` valida los campos obligatorios y evita crear una entidad con datos inválidos.
- `NewLocalStorage(...)` crea las carpetas y recupera los metadatos guardados.
- `NewImageService(...)`, `NewImageHandler(...)` y `NewAPIClient(...)` preparan los componentes que colaboran entre si.
- `Record` incorpora `Metadata` mediante composición; `ImageService` contiene `*LocalStorage` y `ImageHandler` contiene `*ImageService`.

Receptores y encapsulamiento

Los métodos que modifican almacenamiento utilizan receptores por puntero, por ejemplo Save y Delete en LocalStorage. Image mantiene sus campos privados y expone métodos de lectura; como la entidad se trata como un valor de dominio inmutable después de construirse, sus métodos de consulta utilizan receptores por valor.

Manejo de errores

Las funciones devuelven valores de tipo error y los componentes superiores los traducen en respuestas claras. No se utiliza panic para controlar entradas invalidas, archivos inexistentes, formatos no permitidos o problemas habituales de comunicación.

8. Requisitos atendidos y pendientes

Requisito	Estado
Cliente y servidor independientes en Go	Implementado
Comunicación HTTP real y respuestas comprensibles	Implementado
IP, puerto y URL configurables	Implementado
Carga, consulta, búsqueda, descarga y eliminación	Implementado
Metadatos y archivos gestiónados por separado	Implementado
Validación de nombre, formato y tamaño	Implementado
Manejo de datos incompletos y recursos inexistentes	Implementado
Persistencia después de reiniciar el servidor	Implementado
Autenticación integrada mediante el servicio previsto	Fase posterior, no exigida en semana 10
Comunicación entre sedes mediante la infraestructura de red	Fases posteriores

9. Validaciónes y escenarios de error

Situación	Respuesta del prototipo
Datos vacios o nombre inválido	HTTP 400 Bad Request.
Formato no permitido	HTTP 415 Unsupported Media Type.
Archivo superior a 10 MiB	HTTP 413 Request Entity Too Large.
Identificador inexistente	HTTP 404 Not Found.
Metodo no soportado para una ruta	HTTP 405 Method Not Allowed.
Problema de conexión	El cliente informa que no pudo conectarse o enviar la solicitud.

10. Pruebas técnicas ejecutadas

Las siguientes comprobaciones se realizaron en el entorno de verificación del prototipo con Go 1.23.2:

- `go test ./...`: todas las pruebas automatizadas de services, storage y transport finalizaron correctamente.
- `go vet ./...`: no reportó problemas estáticos.
- `go build ./cmd/server` y `go build ./cmd/client`: ambos ejecutables compilaron correctamente.
- Prueba de flujo real: el cliente se conectó al servidor, cargo una imagen PNG, consultó y filtro metadatos, descargo el archivo, recibió un 404 al solicitar un ID inexistente y eliminó la imagen.
- Prueba de formato: el servidor rechazó un archivo TXT con respuesta HTTP 415.

11. Evidencias de ejecución

Inicio del servidor en un puerto de laboratorio configurado para la prueba:

```
ImageFlow Server iniciado en 127.0.0.1:18081
Directorio de datos: /tmp/imageflow-fase2-smoke
Endpoint de prueba: GET /health
API de imágenes: /api/v1/images
```

Salida capturada del cliente al cargar y consultar el archivo de prueba:

```
Conexión establecida correctamente.
Imagen cargada correctamente.
Nombre:      imagen_demo.png
Categoría:   demo
Formato:     image/png
Tamaño:      3698 bytes
Se encontraron 1 imagen(es).
```

Escenarios de descarga y error controlado:

```
Imagen descargada en: /tmp/imageflow-fase2-descarga/imagen_demo.png
Error: el servidor respondió 404 Not Found: imagen no encontrada
Imagen eliminada correctamente.

No se pudo cargar la imagen:
el servidor respondió 415 Unsupported Media Type:
formato de imagen no permitido; use JPG, PNG, GIF o WEBP
```

Nota sobre evidencias: estas salidas corresponden a pruebas reales de verificación del prototipo en el entorno de generación. Para el video y la evidencia personal de entrega, se recomienda repetir la demostración desde el equipo del estudiante y conservar capturas propias.

12. Limitaciones actuales

- Los archivos se almacenan en el disco local del servidor y los metadatos en JSON; no se utiliza una base de datos relacional.
- No se implementa autenticación ni control de roles en esta fase, para no duplicar el servicio de autenticación previsto por la guía.
- La conexión entre dos sedes y su infraestructura de red se implementará en las siguientes fases.
- No se generan miniaturas ni se realiza análisis antivirus.
- El prototipo es una versión inicial academica y no se presenta como un servicio listo para produccion.

13. Enlaces de entrega

Elemento	Enlace
Repositorio Git	[PEGAR URL DEL REPOSITORIO DESPUÉS DE PUBLICARLO]
Video demostrativo	[PEGAR URL DEL VIDEO DESPUÉS DE PUBLICARLO]

14. Autor, contribuciones y uso de IA

Autor: Sebastián Alexis Cumbal Molineros. La presente versión se desarrolla de forma individual. El autor asume el análisis, diseño, programación del cliente y servidor, modelado, validaciónes, pruebas y documentación. La guía de la actividad esta redactada para equipos; por ello, la modalidad individual debe ser comunicada y registrada por el docente.

Se utilizó ChatGPT como herramienta de apoyo para organizar la arquitectura, proponer y revisar código y preparar la documentación. El autor debe revisar, ejecutar, comprender y poder explicar el trabajo entregado. El código del prototipo utiliza Go y su biblioteca estándar; no contiene código fuente en Python ni en otro lenguaje.

15. Conclusión

El prototipo ImageFlow cumple las operaciónes esenciales de la opcion de gestión de imágenes para el segundo entregable: cliente y servidor Go independientes, comunicación HTTP, persistencia diferenciada de archivos y metadatos, validaciónes, búsqueda, descarga, eliminación y manejo de errores. Las pruebas automatizadas y la ejecución de extremo a extremo verificaron el flujo principal. La autenticación y la conectividad entre sedes quedan identificadas como trabajo de fases posteriores, de acuerdo con el alcance de la semana 10.